

Scaling and Convergence Speed

Use your gradient descent implementation to train a multivariate model using untransformed features, and to train another using standardized or normalized features, until they both achieve a positive R^2

Provide a figure showing loss over iterations for both.

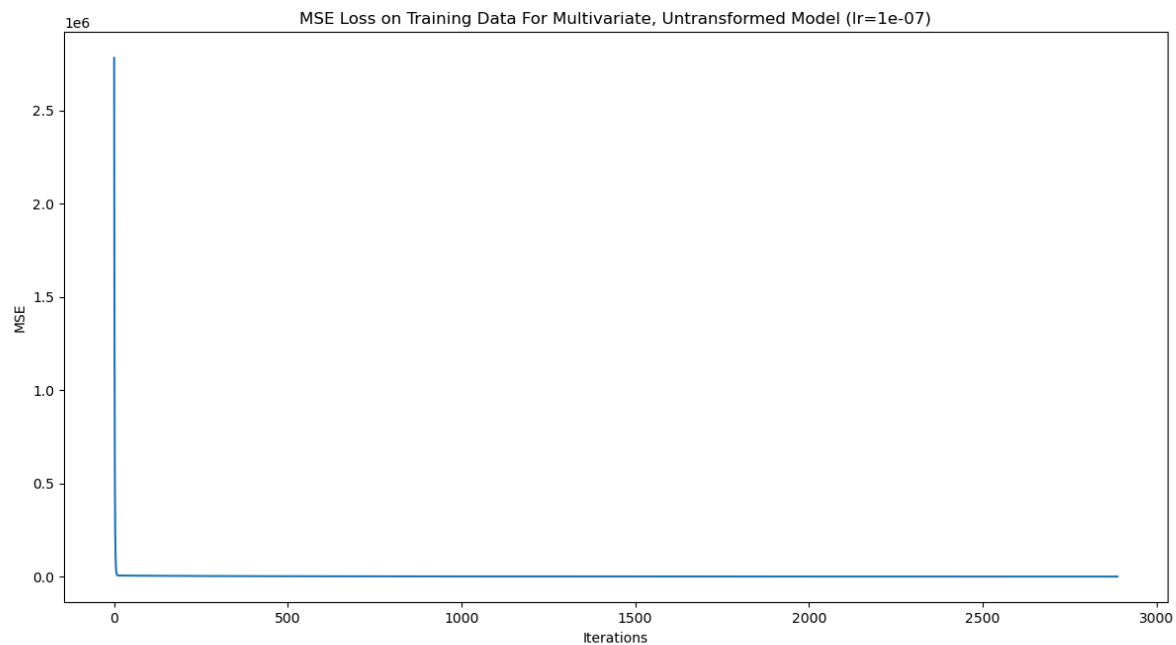


Figure 1: MSE Loss over training epoch for untransformed, linear model with learning rate $1e-7$

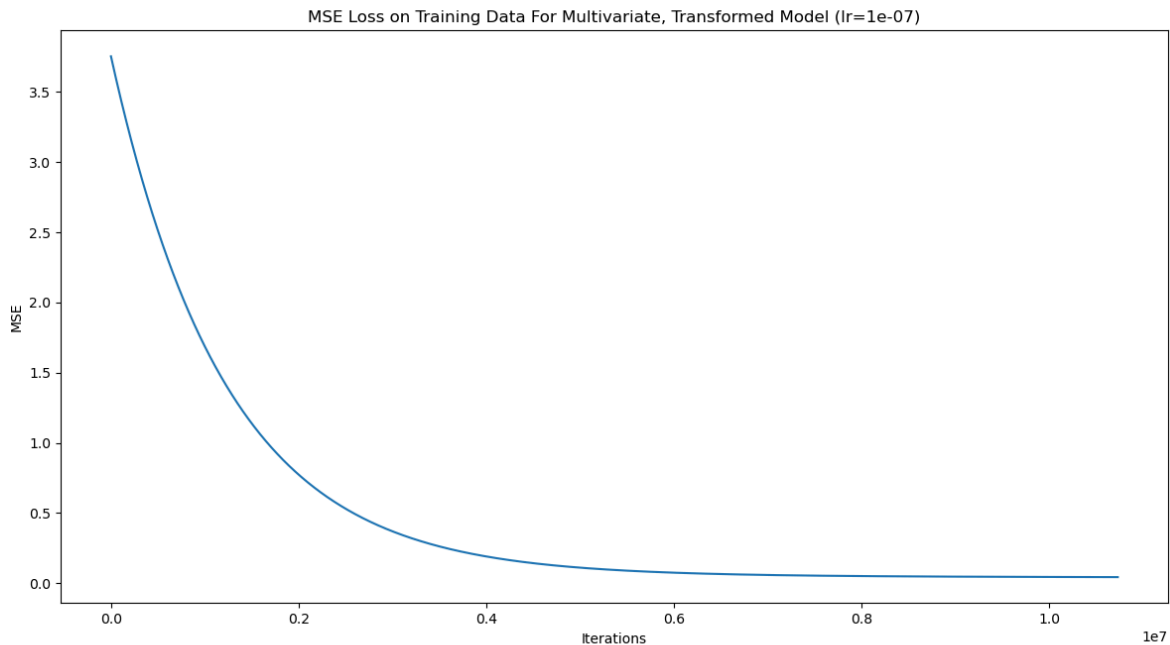


Figure 2: MSE Loss over training epoch for normalized, linear model with learning rate 1e-7

How does feature scaling affect convergence speed and stability? Explain why.

Above a learning rate of roughly 1e-7, the untransformed version did not converge and caused overflow errors. Below this value, the model converged but produced a constant MSE regardless of learning rate (tested up to 1e-10) with the only change being the number of epochs required to get a positive R2 score. The normalized version was able to converge to positive R2 scores for smaller learning rates. Tested at the same learning rate of 1e-7, this model took significantly more epochs to converge but arrived at a much smaller loss once it achieved this.

Scaling and the Optimal Solution

Use an external linear model optimization tool to find the optimal solution for an untransformed multivariate model and a scaled multivariate model.

Does a linear transformation meaningfully change the optimal mathematical solution?

The sklearn.linear_models.LinearRegression library to find the optimal model solutions. Applying the normalization did produce significantly different model weights between the transformed and untransformed versions. However, both produced an identical R2 Score of 0.624 for the training set. This indicates that the weights themselves are only absorbing constant factors due to the normalization applied to the features, keeping the model predictions and the optimal mathematical solution the same once the linear transformation is accounted for.

Non-Linear Transformation

Apply a logarithmic transformation, $\log(x + 1)$, to your features and use your gradient descent implementation to train a new multivariate model.

Provide an updated figure showing loss over iterations.

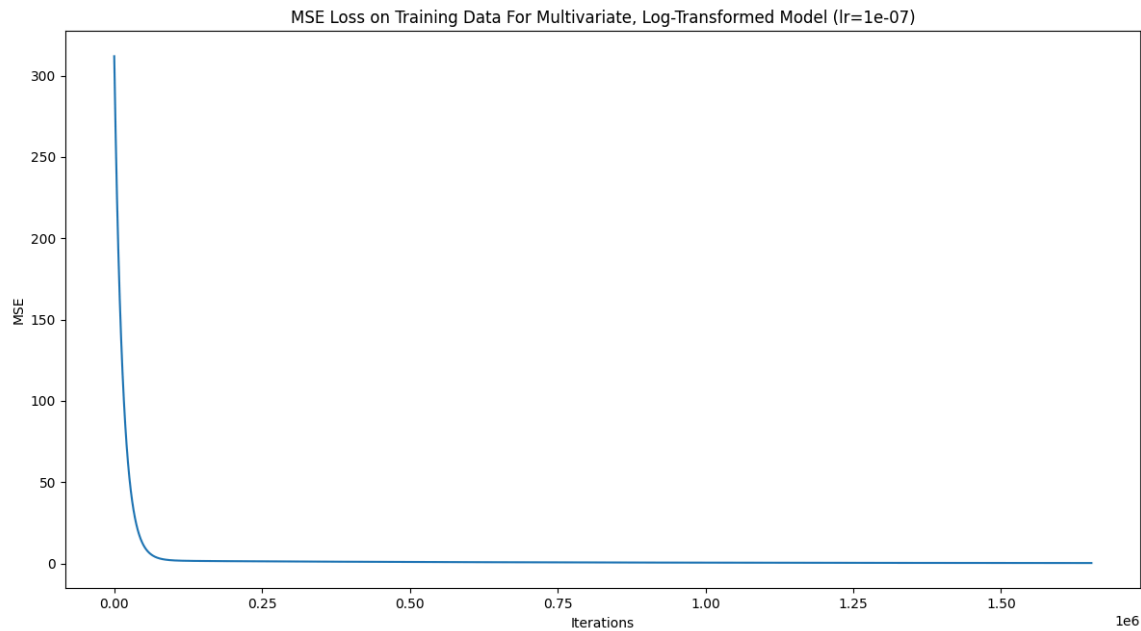


Figure 3: MSE Loss over training epoch for log-transformed, linear model with learning rate 1e-7

How does this non-linear feature transformation affect convergence speed and stability?
After applying this logarithmic transform, the convergence speed appears to be faster than the model trained on normalized features but slower than the model trained on untransformed features. Like the feature-normalized model, this transform has the benefit of always converging to a stable solution since the values are significantly reduced after applying the logarithm.

Construct a summary table comparing Training MSE , Testing MSE , Training R2, and Testing R2 across the three optimal models: Untransformed, Standardized/Normalized, and Log-Transformed.

	Training MSE	Testing MSE	Training R2	Testing R2
Untransformed	276.865	381.834	0.000779	-0.754
Normalized	0.043	85742.150	3.851e-8	-392.784
Log-Transformed	0.268	2505.176	1.305e-6	-10.505

Table 1: Model summary for fixed learning rate = 1e-7 with training stopped at first instance of positive training R2 score

How does a non-linear transformation differ from a linear transformation in its impact on the optimal model solution and overall predictive performance? Explain why.
Regarding the optimal model solution, the log-transformed model appears to find a model that fits the training data worse than the linear-transformed model given it has a higher MSE.

However, the log-transformed model has better predictive performance as its test-set MSE is lower by an order-of-magnitude than the linear-model's test-set MSE. A good MSE on the training set but poor test-set predictive performance indicates that the linear-transformed model is overfitting to the training data. Additionally, the nonlinear transformation producing a model with better predictive performance indicates that the data is likely to have a nonlinear relationship between some of the features and compressive strength.

Univariate and Multivariate Coefficients

Use your gradient descent implementation to train eight separate univariate models $y = m \cdot x + b$ for every untransformed predictor until you achieve a positive R^2 for at least two of the eight models.

Which models were easy to train? Which models were hard?

The models using coarse and fine aggregate features were hard to train, taking roughly 10 million iterations before converging at a positive R^2 score. Age and superplasticizer were less difficult, taking only 1 million iterations. Cement was the easiest, taking only 15 iterations before convergence.

Use an external linear model optimization tool to find the optimal solution for each of the univariate models. Compare the individual coefficients from these univariate models to the corresponding feature weights found in the optimal untransformed multivariate model.

Are they identical? Support your answer with a table of coefficients.

As shown in Table 2, the coefficients are not identical.

Feature	Multivariate Coefficient (slope,intercept)	Univariate Coefficient (slope,intercept)
Cement (0)	0.132845	0.0808
Blast Furnace Slag (1)	0.12504038	0.0312
Fly Ash (2)	0.10673935	-0.0533
Water (3)	-0.13313546	-0.206
Superplasticizer (4)	0.11618198	0.883
Coarse Aggregate (5)	0.02863288	-0.0274
Fine Aggregate (6)	0.0340842	-0.0306
Age (7)	0.11876335	0.0950

Table 2: Multivariate vs. univariate feature coefficients for SciKit LinearRegression models

Why are univariate coefficients not equal to multivariate coefficients? Explain the

roles of feature correlations and omitted variable bias.

We would expect that the univariate and multivariate coefficients are identical only if the features are all independent. Features are independent if they are uncorrelated, which cannot be true in this data. One example could be that the amount of aggregate is correlated to the amount of cement, splitting the two features' contribution to compressive strength between their weights. Another perspective is to consider that a univariate model, predicting with only one feature, is unlikely to fully describe the data. This creates an omitted variable bias in the weights, since they must account for the effect the single feature had on the data as well as compensating for the unrecognized features.

Statistical Significance

Use a statistical analysis package (e.g., statsmodels.api.OLS) to extract feature p -values.

Present a consolidated table listing all features across three configurations:

- Untransformed Multivariate Model p -values
- Standardized/Normalized Multivariate Model p -values
- Log-Transformed Multivariate Model p -values

Feature	Untransformed	Normalized	Log-Transformed
Constant	7.082e-02	1.283e-01	4.838e-01
Cement	6.971e-41	6.971e-41	1.039e-88
Blast Furnace Slag	4.145e-26	4.145e-26	2.000e-33
Fly Ash	1.895e-13	1.895e-13	1.676e-05
Water	1.918e-03	1.918e-03	1.765e-11
Superplasticizer	2.461e-01	2.461e-01	6.670e-06
Coarse Aggregate	5.443e-03	5.443e-03	1.037e-01
Fine Aggregate	2.738e-03	2.738e-03	2.223e-01
Age	9.516e-74	9.516e-74	3.741e-198

Table 3: p -values for features in untransformed, linear-transformed, and log-transformed linear regression models

Are there any trends between which predictors are important, and which univariate models were easy or hard to train?

Yes. The models that were easier to train (cement and age) had the smallest p -values. The models that were the hardest to train (coarse and fine aggregate) had p -values which were the largest. This can be understood as the training process having an easier time finding solutions for features that have more importance.

Which transformation type drastically changes the statistical significance of the

predictors, and which does not? Explain why.

The logarithmic transformation changed the statistical significance while the linear-normalization had no effect, relative to the untransformed scores. The unnormalized and linear-normalized have the same statistical significance since the t-test standardizes the data against its mean and variance; a linear transformation of the data would not affect its standardized value, while a nonlinear transformation, like the logarithm, would change the standardized data and therefore change the p-value derived from it.